

ENGINEERING REFERENCE · V2.0.0

How the Limitless backend actually works

One document that walks through every real flow — signing up, verifying, taking an assessment, getting a report, paying, and administering the platform — plus a complete endpoint reference at the end.

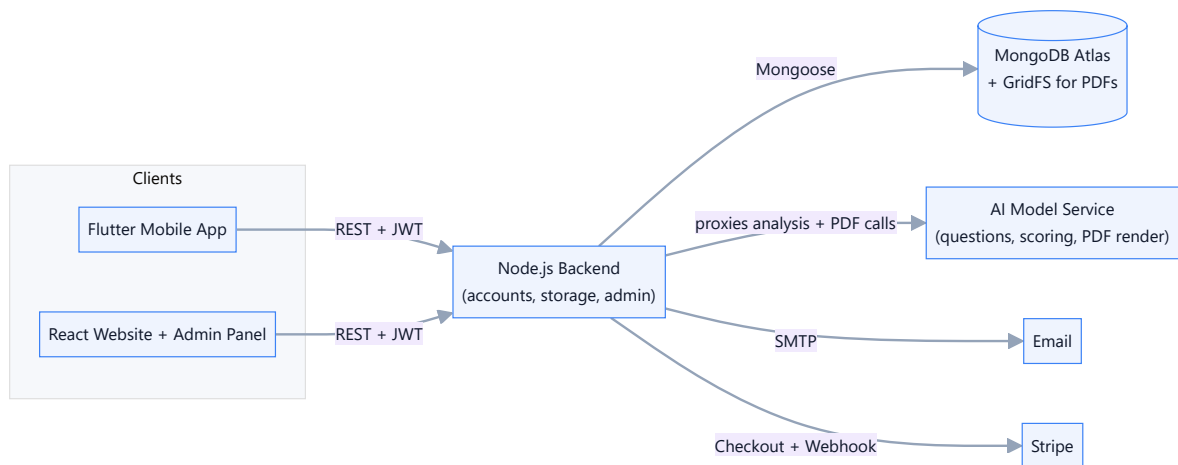
NODE BACKEND `limitless-api.160-153-179-249.sslip.io`

AI MODEL SERVICE `limitless-model.160-153-179-249.sslip.io`

01 Two services, one app

Everything the app talks to is either the Node backend or the AI model service — never anything else. Keeping that split straight explains most "why does this look like the wrong thing" bugs.

The **Node.js backend** owns accounts, sessions, stored assessments, admin tooling, payments, and file storage. The **AI model service** is a separate Python service — the only place questions get generated, answers get scored, and PDFs get rendered. The Node backend never re-implements any of that; where it needs a PDF, it calls the model service itself and stores the result.



The app calls the AI model service directly for questions and scoring, but always goes through the Node backend to generate and store a PDF — that's what keeps a report tied to the right assessment and behind the paywall.

02 Account lifecycle

This is the flow that's changed the most, so it's worth reading slowly. The short version: **registration comes first and never needs an OTP**; verification is something that happens afterward and never blocks anything.

1

Register — `POST /api/auth/register`

Body: name, email, age, gender, and an optional password. Nothing else is required first. The account is created immediately and the response includes a ready-to-use token — the app can log the user straight in without a separate login call.

2

Password — chosen by the user, or generated

Send password (6+ chars) if the screen collects one — it becomes the real password immediately, and no reset is forced. Omit it and the backend generates an 8-character temporary one, emails it, and returns it once as `tempPassword`; that account gets `password_reset_required: true`.

3

Verify the email — `POST /api/auth/send-otp` then `POST /api/auth/verify-otp`

Called *after* registration, on the same email. A 6-digit code is emailed (10 min to use it, 60s between resends). Verifying flips `email_verified` to `true` on the account —

it's a status flag, not a gate. The user can use the app before, during, or without ever completing this step.

4

Return visits — `POST /api/auth/login`

For a session that's been signed out or a fresh install: `email` + whichever password is currently active. Returns a new token plus the user's `latestAssessment`.

5

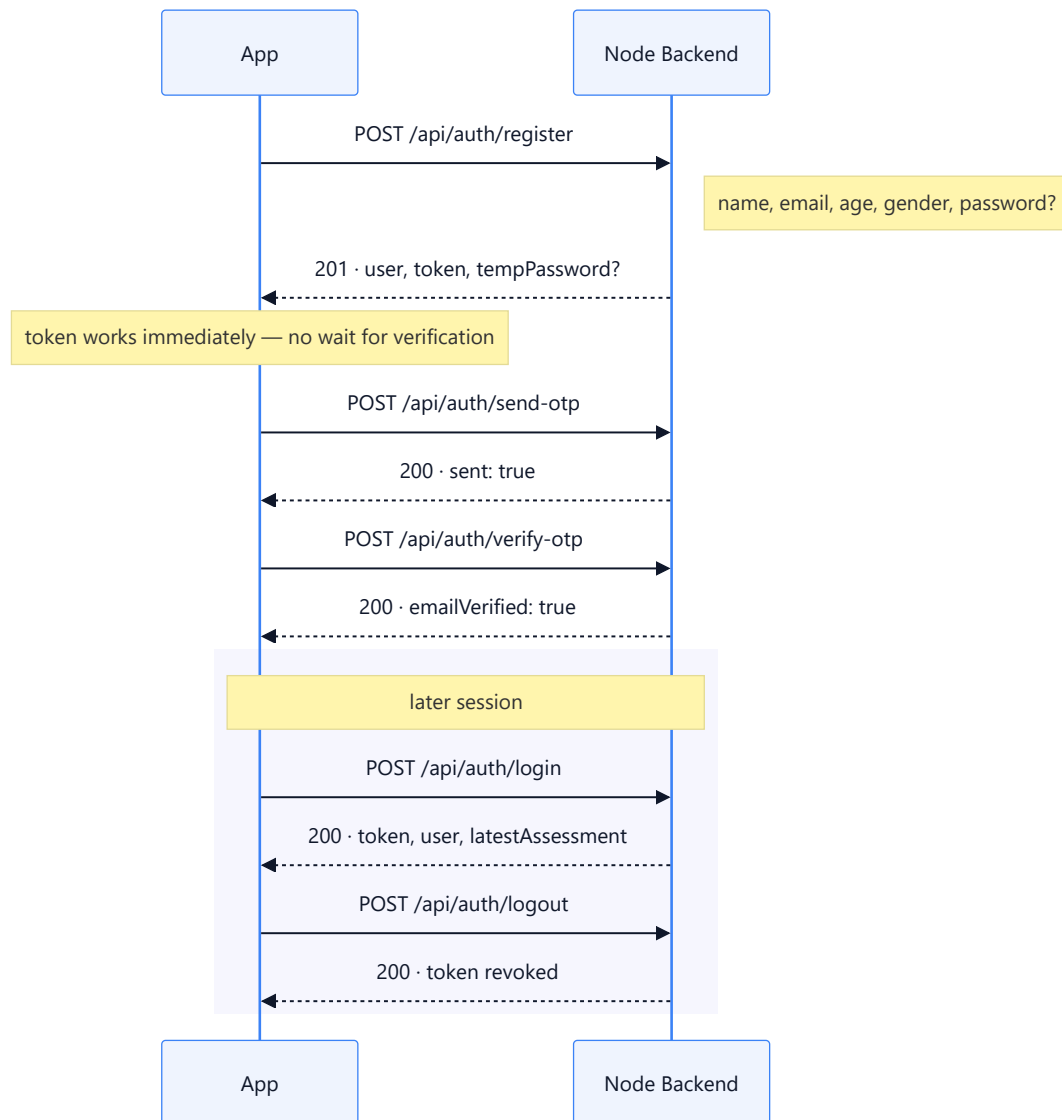
Signing out — `POST /api/auth/logout`

Sends the current token to a server-side revocation list — it stops working immediately, not just when the app forgets it. Call this on the actual sign-out action.

6

Forgot password — `POST /api/auth/forgot-password` → `POST /api/auth/reset-password`

For an existing account that's lost its password. Emails a reset code, then `email` + `otp` + `newPassword` sets a new one directly — no need to know the old one.

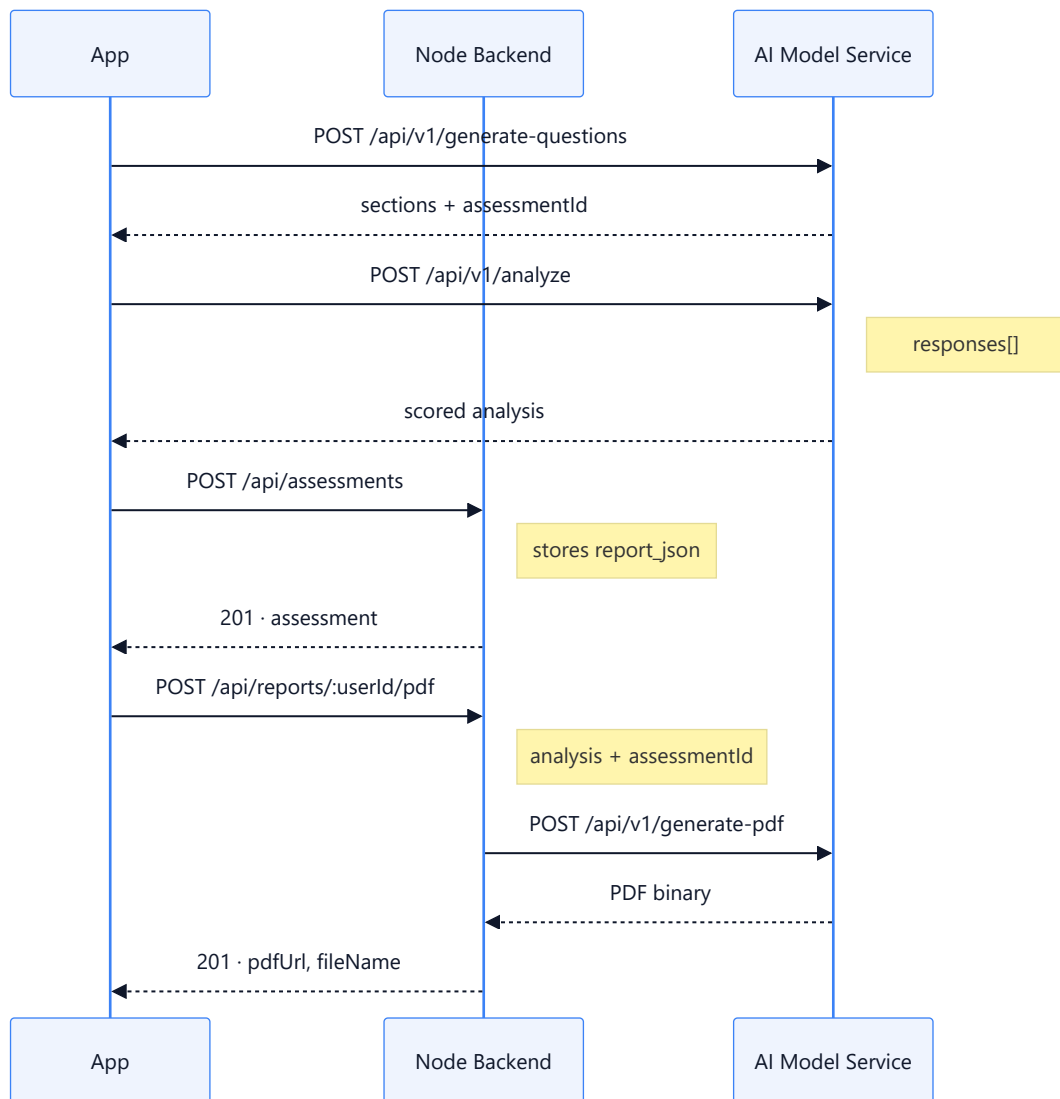


Registration and verification are independent — an unverified user is a fully functional user.

! Don't send a raw password to `/register` and expect old behavior — if you omit it, one is generated for you. And don't call `/send-otp` before registering; it's meant for a user that already exists.

03 Taking an assessment, getting a report

Question generation and scoring talk to the AI model service directly. Turning a scored result into a stored, downloadable PDF goes through the Node backend, which enforces the paywall and keeps every assessment's report separate.



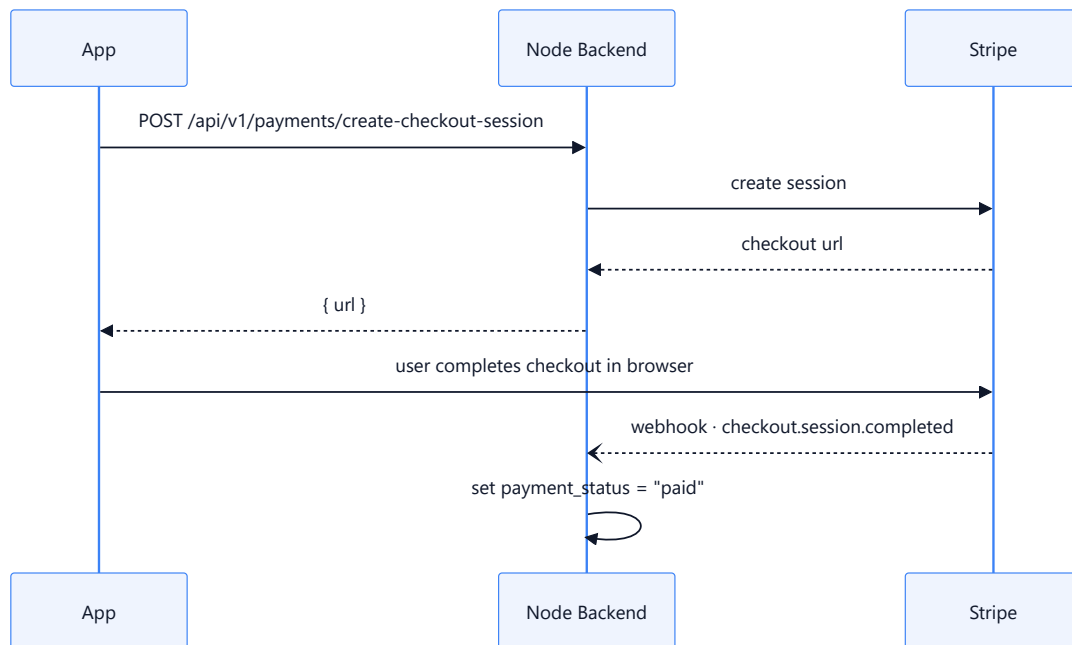
The `assessmentId` passed to the PDF call matters — it's what keeps a user's second assessment from overwriting their first one's report.

The paywall, enforced server-side

A full (non-teaser) PDF only generates for a user whose `payment_status` is `paid` — the backend checks this itself and returns `403` otherwise, regardless of what the app's UI allows. Pass `teaser: true` for the free preview report, which skips that check entirely.

04 Payments

The redirect back into the app after checkout is not what marks a user paid — the Stripe webhook is, and it's signature-verified so nothing else can fake it.



If the app polls `GET /api/users/:id` after redirect, it's really waiting on that webhook to land — usually a second or two.

05 Admin panel

A separate login and a separate 12-hour token — an admin token can act on any user's data; a user token can only ever act on its own.

Once signed in with `POST /api/admin/login`, the panel lists every user with their assessments and credentials, edits a user's plan directly (`paid`, `demo`, `suspended`, etc. — this is the only way `payment_status` changes outside of a real Stripe payment), reviews contact-form enquiries, and reads aggregate stats — total users, conversion rate, MRR.

06 Endpoint reference

Every route, grouped the way the flows above use them. **open** needs no token; everything else needs **Authorization: Bearer <token>**.

METHOD	PATH	AUTH	WHAT IT DOES
<i>Health</i>			
GET	/health	open	Service status — also at / and /api/v1/health
<i>Auth · Node backend</i>			
POST	/api/auth/register	open	Create an account (password optional)
POST	/api/auth/send-otp	open	Email a verification code, post-registration
POST	/api/auth/verify-otp	open	Verify the code, sets email_verified
POST	/api/auth/login	open	Returning-user sign-in
POST	/api/auth/logout	user token	Server-side token revocation
POST	/api/auth/change-password	open *	Set a new password, knowing the old one
POST	/api/auth/forgot-password	open	Email a reset code to an existing account
POST	/api/auth/reset-password	open *	Set a new password using the reset code
<i>Users & assessments</i>			
GET	/api/users/:id	self or admin	Profile + assessment history
PATCH	/api/users/:id	self or admin	Update profile (payment_status is admin-only)
POST	/api/assessments	self or admin	Store a scored report against a user
PATCH	/api/assessments/:id	owner or admin	Update a stored report or its pdf_url

METHOD	PATH	AUTH	WHAT IT DOES
<i>AI model service (different base URL)</i>			
POST	/api/v1/generate-questions	open	Build a question set for an age/gender
POST	/api/v1/analyze	open	Score a set of responses
POST	/api/v1/generate-pdf	open	Render the full report PDF (usually called by the Node backend, not directly)
POST	/api/v1/generate-teaser-pdf	open	Render the free teaser PDF
POST	/api/v1/longitudinal-analysis	open	Compare a user's assessments over time
<i>Reports & files · Node backend</i>			
POST	/api/reports/:userId/pdf	self or admin	Generate + store a report PDF, paywall-checked
GET	/files/pdf-reports/:userId/:fileName	open	Download a stored PDF
POST	/api/files/pdf-reports/:userId	self or admin	Upload a raw PDF directly
<i>Payments</i>			
POST	/api/v1/payments/create-checkout-session	user token	Start a Stripe Checkout session
POST	/api/v1/webhooks/stripe	Stripe only	Signature-verified payment confirmation
<i>Plans & enquiries</i>			
GET	/api/plans	open	List active pricing plans
POST	/api/enquiries	open	Submit the public contact form
<i>Admin</i>			
POST	/api/admin/login	open *	Admin sign-in, 12h token
GET	/api/admin/users	admin	Every user, with assessments + credentials

METHOD	PATH	AUTH	WHAT IT DOES
GET	/api/admin/users/:id	admin	Single user detail
PUT	/api/admin/users/:id/plan	admin	Change a user's payment_status
DELETE	/api/admin/users/:id	admin	Delete a user + their assessments/PDFs
GET	/api/admin/enquiries	admin	List contact-form submissions
DELETE	/api/admin/enquiries/:id	admin	Remove one
GET	/api/admin/stats	admin	Aggregate counts, MRR, conversion rate

* requires credentials in the body (current password, reset code, or admin password) rather than a bearer token.